УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 1

УСТАНОВКА ЯЗЫКА ПРОГРАММИРОВАНИЯ PYTHON. СОЗДАНИЕ ПРОСТЕЙШИХ ЛИНЕЙНЫХ ПРОГРАММ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Установка языка программирования Python. Создание простейших линейных программ

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Алгоритм. Свойства алгоритма. Способы описания алгоритма

Если нужно написать программу на каком-либо языке программирования, то с начало следует составить алгоритм решения задачи.

Алгоритм — это точное и простое описание последовательности действии для решения данной задачи. Алгоритм содержит несколько шагов, которые должны выполняться в определенной последовательности. Каждый шаг алгоритма может состоять из одной или нескольких простых операций.

Каждый из нас ежедневно использует различные алгоритмы: инструкции, правила, рецепты и т. д. Обычно мы это делаем не задумываясь. Например, открывая дверь ключом, никто не размышляет над тем, в какой последовательности выполнять действия. Однако, чтобы кого-нибудь научить открывать дверь, придется четко указать и сами действия, и порядок их выполнения. Например:

- достать ключ;
- вставить ключ в замочную скважину;
- повернуть ключ два раза против часовой стрелки;
- вынуть ключ.

Представим, что мы поменяли местами второе и третье действия. Мы сможем выполнить и этот алгоритм, но дверь не откроется, т. е. алгоритм станет невыполнимым.

Для алгоритма важен не только набор действий, но и то, в каком порядке они выполняются. Понятие алгоритма в информатике является фундаментальным.

Таким же, какими являются понятия точки, прямой и плоскости в геометрии, вещества в химии, пространства и времени в физике и т. д.

Свойства алгоритма:

– дискретность (прерывность, раздельность) алгоритм должен представлять процесс решения задачи как последовательное выполнение простых шагов (этапов);

– определенность каждый шаг алгоритма должен быть четким и однозначным. Выполнение алгоритма носит механический характер и не требует никаких дополнительных сведений о решаемой задаче;

– результативность - алгоритм должен приводить к решению задачи за конечное число шагов;

– массовость алгоритм решения разрабатывается в общем виде. т. с. он должен быть применим для решения некоторого класса задач, различающихся лишь исходными данными.

Способы описания алгоритмов:

– словесный;
– графический;
– табличный;
– формульный.

Словесный способ каждый использует ежедневно, пересказывая собеседнику, например, различные инструкции, правила, кулинарные рецепты. т. е. какую-то последовательность, приводящую к конечному результату.

Графический способ представления алгоритмов является более компактным и наглядным по сравнению со словесным. Как часто для лучшего понимания той или иной ситуации проще начертить какую-то схему, план, согласно которым необходимо будем действовать. В программировании данный способ предпочтителен. поскольку позволяет с помощью последовательности функциональных блоков, каждый из которых соответствует выполнению одного или нескольких действий, представить ход решения той или иной задачи. Такое представление алгоритма называется структурной схемой алгоритма, или блок-схемой.

Табличный способ используется, например, в бухгалтерии при составлении ежегодных отчетов. сводок и т. д.

Формульный способ находит свое применение при решении задач из области математики, физики и т. д. Например, при решении квадратного уравнения приступают к нахождению дискриминанта уравнении, а затем, в зависимости от полученного результата, находят корни уравнения по известным всем формулам.

4.2 Назначение функциональных блоков

На условные обозначения в схемах алгоритмов, программ, данных и систем распространяется ГОСТ 19.701—90. «ЕСПД. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения».

При отрисовке блок-схем мы будем использовать функциональные блоки, представленные ниже. Заметим, что некоторые их названия или отрисовка не всегда совпадают с указанными в ГОСТ, поэтому при создании схем алгоритмов

для курсовых, дипломных работ, безусловно, следует обратиться к существующему стандарту. При отрисовке блок-схем используются функциональные блоки, представленные на рисунке 1.

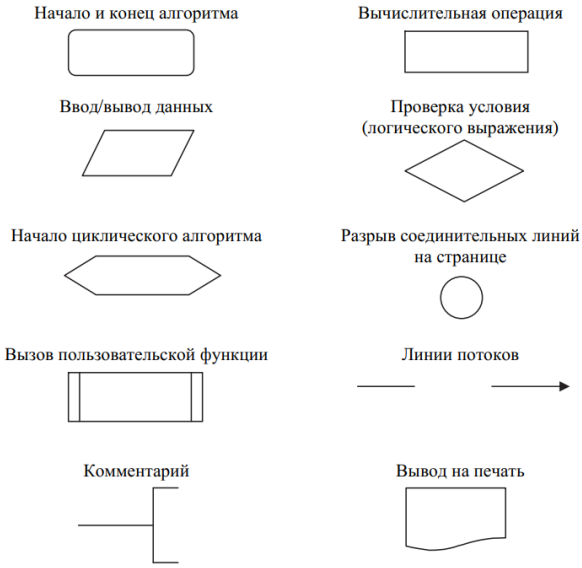


Рисунок 1 – Функциональные блоки при отрисовке блок-схем

4.3 Основные этапы решения задач

Процесс решения некоторой задачи в среде программирования, как правило, состоит из нескольких этапов, часть из которых выполняется пользователем, а часть компьютером.

1- й этап. Общая постановка задачи.

На этом этапе описывается содержание задачи, составляется перечень исходных данных.

2- й этап. Разработка математической модели.

Цель этого этапа состоит в установлении формализованных связей между исходными данными и искомыми результатами. Этап заключается в записи расчетных формул или функциональных зависимостей.

3- й этап. Разработка алгоритма.

Этап состоит в описании последовательности действий, в результате которых может быть получено решение задачи.

4- й этап. Разработка программы.

Программа составляется в полном соответствии с разработанным алгоритмом решения задачи.

5- й этап. Отладка программы.

Процесс поиска ошибок в программе и их устранение.

6- й этап. Анализ результатов.

Позволяет принять решение о необходимости внесения изменений в программу, проведении дополнительных расчетов или их окончании.

4.4 Алфавит языка Python

Изучение любого языка начинается с изучения алфавита, из букв складываются слова, из слов предложения. То же происходит и при изучении языка программирования. Сначала необходимо уяснить, какие символы можно использовать для записи слов языка, из которых можно формировать определенные конструкции. Итак, в алфавит языка Python входят:

– латинские буквы от a до z и от A до Z;

В Python есть различия между прописными п строчными буквами алфавита, например, chislo, CHISLO, Chislo разные имена переменных.

– цифры от 0 до 9;
– специальные символы, например +, -, *, /;
– зарезервированные (служебные) слова: for, if, class, def и т. д.

4.5 Идентификаторы и общие правила их написания

Для того чтобы программа решения задачи обладала свойством массовости, следует употреблять не конкретные значения величин, а использовать их обозначения для возможности изменения по ходу выполнения программы их значений. Для обозначения в программе переменных и постоянных величин используются имена идентификаторы *(identification* установление соответствия объекта некоторому набору символов).

Программа на Python представляет собой последовательность инструкций, которые называются операторами. Необходимо учитывать, следующее:

– в идентификатор не могут входить пробелы, специальные символы алфавита;
– идентификатор начинается только с буквы или со знака подчеркивания;
– идентификатор может состоять из букв, цифр и знака подчеркивания;
– при написании идентификатора можно использовать как прописные, так и строчные буквы латинскою алфавита;
– идентификатор не должен являться зарезервированным словом.

4.6 Комментарии в языке Python

Комментарии предназначены для вставки пояснений в текст программы, интерпретатор полностью их игнорирует. Внутри комментария может располагаться любой текст, включая инструкции, которые выполнять не следует.

В языке **Python** присутствует только ***однострочный комментарий***. Он начинается с символа #:

```
# Это комментарий
```

Однострочный комментарий может начинаться не только с начала строки, но и располагаться после инструкции. Например, в следующем примере

комментарий расположен после инструкции, предписывающей вывести надпись "Привет, мир!":

```
print("Привет, мир!") # Выводим надпись с помощью функции print()
```

Если же символ комментария разместить перед инструкцией, то она не будет выполнена:

```
# print("Привет, мир!") Эта инструкция выполнена не будет
```

Если символ # расположен внутри кавычек или апострофов, то он не является символом комментария:

```
print("# Это НЕ комментарий")
```

Так как в языке Python нет многострочного комментария, то часто комментируемый фрагмент размещают внутри утроенных кавычек (или утроенных апострофов):

```
"""
Эта инструкция выполнена не будет
print ("Привет, мир!")
"""
```

Следует заметить, что этот фрагмент кода не игнорируется интерпретатором, т. к. он не является комментарием. В результате выполнения фрагмента будет создан объект строкового типа. Тем не менее инструкции внутри утроенных кавычек выполнены не будут, поскольку интерпретатор сочтет их простым текстом. Такие строки являются строками документирования, а не комментариями.

4.7 Ввод данных в языке Python

Для ввода данных в Python 3 предназначена функция **input ()**, которая получает данные со стандартного ввода **stdin**. Функция имеет следующий формат:

```
[<Значение> = ] input([<Сообщение>])
```

Для примера переделаем первую программу так, чтобы она здоровалась не со всем миром, а только с нами (рисунок 2).

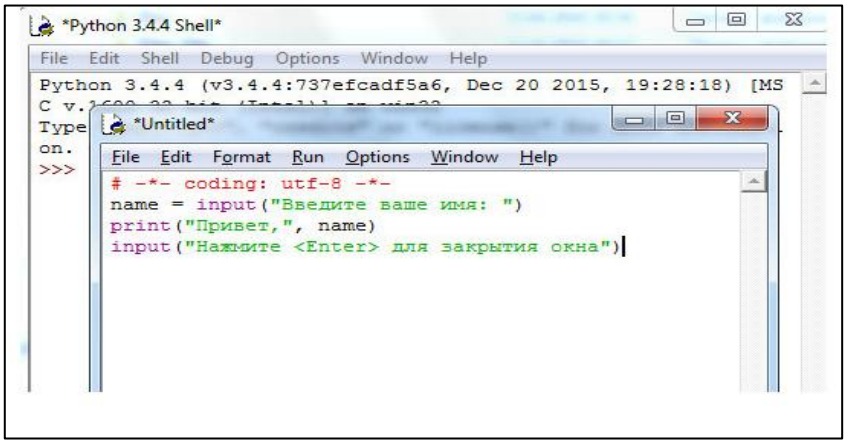


Рисунок 2 – Пример использования функции input ()

Чтобы окно сразу не закрылось, повторно вызываем функцию **input ()**. В этом случае окно не закроется, пока не будет нажата клавиша **Enter**.

Функция **input ()** позволяет вводить выражения, которые затем, с помощью функции **eval ()**, можно выполнить. Например, следующая программа запрашивает выражение и вычисляет его (рисунок 3):

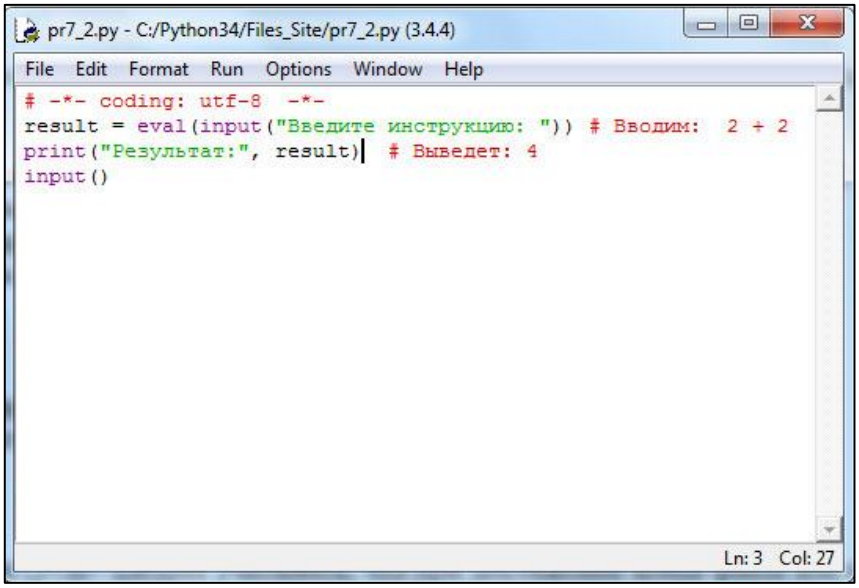


Рисунок 3 – Использование функции eval ()

4.8 Вывод результатов работы программы в языке Python

Вывести результаты работы программы можно с помощью функции **print ()**. Функция имеет следующий формат:

print([<Объекты>][, sep=' '][, end='\n'][, file=sys.stdout][, flush=False])

Функция **print ()** преобразует объект в строку и посылает ее в стандартный вывод **stdout**. С помощью параметра **file** можно перенаправить вывод в другое место - например, в файл. При этом, если параметр **flush** имеет значение **False**, выводимые значения будут принудительно записаны в файл. После вывода строки автоматически добавляется символ перевода строки:

```
print ("Строка 1")
print ("Строка 2")
```

Результат:

```
Строка 1
Строка 2
```

Если необходимо вывести результат на той же строке, то в функции **print ()** данные указываются через запятую:

```
print ("Строка  1", "Строка 2")
```

Результат:

```
Строка 1 Строка 2
```

Как видно из примера, между выводимыми строками автоматически вставляется пробел. С помощью параметра **sep** можно указать другой символ. Например, выведем строки без пробела между ними:

```
print ("Строка 1", "Строка 2", sep="")
```

Результат:

```
Строка 1Строка 2
```

После вывода объектов в конце добавляется символ перевода строки. Если необходимо произвести дальнейший вывод на той же строке, то в параметре **end** следует указать другой символ:

```
print ("Строка 1", "Строка 2", end=" ")
print ("Строка 3")
#  Выведет: Строка  1 Строка 2  Строка  3
```

Для вывода результатов работы программы вместо функции **print ()** можно использовать метод **write ()** объекта **sys.stdout**:

```
import sys                  # Подключаем модуль sys
sys.stdout.write ("Строка") # Выводим строку
```

4.9 Типы данных в языке Python

В **Python 3** объекты могут иметь следующие типы данных:

– **bool** – логический тип данных. Может содержать значения **True** или **False**, которые ведут себя как числа 1 и 0 соответственно:

```
print(type(True), type(False))
(<class 'bool'>, <class 'bool'>)
```

– **NoneType** – объект со значением **None** (обозначает отсутствие значения):

```
print(type(None))
<class 'NoneType'>
```

В логическом контексте значение **None** интерпретируется как **False**:

```
print(bool(None))
False
```

– **int** – целые числа. Размер числа ограничен лишь объемом оперативной памяти:

```
print(type(2147483647), type(99999999999999999999999999999))
(<class 'int'>, <class 'int'>)
```

– **float** - вещественные числа:

```
print(type(5.1), type(8.5e-3))
(<class 'float'>, <class 'float'>)
```

– **complex** - комплексные числа:

```
print(type(2 + 2j))
<class 'complex'>
```

– **str** - Unicode-строки:

```
print(type('Строка'))
<class 'str'>
```

– **bytes** - неизменяемая последовательность байтов:

```
print(type(bytes('Строка', 'utf-8')))
<class 'bytes'>
```

– **bytearray -** изменяемая последовательность байтов:

```
print(type(bytearray('Строка', 'utf-8')))
<class 'bytearray'>
```

– **list -** списки. Тип данных list аналогичен массивам в других языках программирования:

```
print(type([1, 2, 3]))
<class 'list'>
```

– **tuple -** кортежи:

```
print(type((1, 2, 3)))
<class 'tuple'>
```

– **range -** диапазоны:

```
print(type(range(1, 10)))
<class 'range'>
```

– **dict -** словари. Тип данных dict аналогичен ассоциативным массивам в других языках программирования:

```
print(type({'x': 5, 'y': 20}))
<class 'dict'>
```

– **set -** множества (коллекции уникальных объектов):

```
print(type({'a', 'b', 'c'}))
<class 'set'>
```

– **frozenset -** неизменяемые множества:

```
print(type(frozenset(['a', 'b', 'c'])))
<class 'frozenset'>
```

– **function -** функции:

```
def func(): pass
print(type(func))
<class 'function'>
```

– **module -** модули:

```
import sys
print(type(sys))
<class 'module'>
```

– **type -** классы и типы данных. Все данные в языке Python являются объектами, даже сами типы данных!

```
class C: pass
print(type(C))
<class 'type'>
print(type(type('')))
<class 'type'>
```

Основные типы данных делятся на **изменяемые** и **неизменяемые**. К **изменяемым типам** относятся:

- **списки**;
- **словари**;

- **тип bytearray**.

Пример изменения элемента списка:

```
arr = [1, 2, 3]
arr[0] = 0
print(arr)
[0, 2, 3]
```

К **неизменяемым типам** относятся:

- **числа**;
- **строки**;
- **кортежи**;
- **диапазоны**;
- **тип bytes**.

Например, чтобы получить строку из двух других строк, необходимо использовать операцию конкатенации, а ссылку на новый объект присвоить переменной:

```
str1 = 'авто'
str2 = 'транспорт'
str3 = str1 + str2
print(str3)
автотранспорт
```

Кроме того, типы данных делятся на **последовательности** и **отображения**. К **последовательностям** относятся:

- строки;
- списки;
- кортежи;
- диапазоны;
- типы bytes и bytearray,

А к отображениям - словари.

4.10 Присваивание значения переменным в языке Python

В языке **Python** используется ***динамическая типизация***. Это означает, что при присваивании переменной значения интерпретатор автоматически относит переменную к одному из типов данных. Значение переменной присваивается с помощью оператора = таким образом:

```
x = 7 # Тип int
y = 7.8 # Тип float
s1 = "Строка" # Переменной s1 присвоено значение Строка
s2 = 'Строка' # Переменной s2 тоже присвоено значение Строка
b = True      # Переменной b присвоено логическое значение True
```

В одной строке можно присвоить значение сразу нескольким переменным:

```
x = y = 10
x, y
(10, 10)
```

После присваивания значения в переменной сохраняется **ссылка на объект, а не сам объект**. Это обязательно следует учитывать при **групповом присваивании**. Групповое присваивание можно использовать для чисел, строк и кортежей, но для изменяемых объектов этого делать нельзя. Пример:

```
x = y = [1, 2] # Якобы создали два объекта
x, y
([1, 2], [1, 2])
```

В этом примере создан список из двух элементов и присвоено значение переменным **x** и **y**. Теперь попробуем изменить значение в переменной **y**:

```
y[1] = 100 # Изменяем второй элемент
x, y
([1, 100], [1, 100])
```

Как видно из примера, изменение значения в переменной **y** привело также к изменению значения в переменной **x**. Таким образом, обе переменные ссылаются на один и тот же объект, а не на два разных объекта. Чтобы получить два объекта, необходимо производить раздельное присваивание:

```
x = [1, 2]
y = [1, 2]
y[1] = 100 # Изменяем второй элемент
x, y
([1, 2], [1, 100])
```

Проверить, ссылаются ли две переменные на один и тот же объект, позволяет оператор **is**. Если переменные ссылаются на один и тот же объект, то оператор **is** возвращает значение **True**:

```
x = y = [1, 2] # Один объект
x is y
True
x = [1, 2] # Разные объекты
y = [1, 2] # Разные объекты
x is y
False
```

Следует заметить, что в целях повышения эффективности кода интерпретатор производит кэширование малых целых чисел и небольших строк. Это означает, что если ста переменным присвоено число 2, то в этих переменных будет сохранена ссылка на один и тот же объект. Пример:

```
x = 2; y = 2; z = 2
x is y, y is z
(True, True)
```

Посмотреть количество ссылок на объект позволяет метод **getrefcount ()** из модуля **sys**:

```
import sys # Подключаем модуль sys
sys.getrefcount(2)
472
```

Когда число ссылок на объект становится равно нулю, объект автоматически удаляется из оперативной памяти. Исключением являются объекты, которые подлежат кэшированию.

Помимо группового присваивания, язык **Python** поддерживает **позиционное присваивание**. В этом случае переменные указываются через запятую слева от оператора =, а значения - через запятую справа. Пример позиционного присваивания:

```
x, y, z = 1, 2, 3
x, y, z
(1, 2, 3)
```

С помощью позиционного присваивания можно поменять значения переменных местами. Пример:

```
x, y = 1, 2; x, y
(1, 2)
x, y = y, x; x, y
(2, 1)
```

По обе стороны оператора = могут быть указаны последовательности. Напомним, что к последовательностям относятся строки, списки, кортежи, диапазоны, типы **bytes** и **bytearray**. Пример:

```
x, y, z = '123' # Строка
x, y, z
('1', '2', '3')
x, y, z = [1, 2, 3] # Список
x, y, z
(1, 2, 3)
x, y, z = (1, 2, 3) # Кортеж
x, y, z
(1, 2, 3)
[x, y, z] = (1, 2, 3) # Список слева, кортеж справа
x, y, z
(1, 2, 3)
```

Количество элементов справа и слева от оператора = должно совпадать, иначе будет выведено сообщение об ошибке:

```
x, y, z = (1, 2, 3, 4)
Traceback (most recent call last):
  File "<pyshell#50>", line 1, in <module>
    x, y, z = (1, 2, 3, 4)
ValueError: too many values to unpack (expected 3)
```

Python 3 при несоответствии количества элементов справа и слева от оператора = позволяет сохранить в переменной список, состоящий из лишних элементов. Для этого перед именем переменной указывается звездочка (*). Пример:

```
x, y, *z = (1, 2, 3, 4)
x, y, z
(1, 2, [3, 4])
```

```
x, *y, z = (1, 2, 3, 4)
x, y, z
(1, [2, 3], 4)
*x, y, z = (1, 2, 3, 4)
x, y, z
([1, 2], 3, 4)
x, y, *z = (1, 2, 3)
x, y, z
(1, 2, [3])
x, y, *z = (1, 2)
x, y, z
(1, 2, [])
```

Как видно из примера, переменная, перед которой указана звездочка, всегда содержит список. Если для этой переменной не хватило значений, то ей присваивается пустой список. Следует помнить, что звездочку можно указать только перед одной переменной. В противном случае возникнет неоднозначность и интерпретатор выведет сообщение об ошибке:

```
*x, y, *z = (1, 2, 3, 4)
SyntaxError: two starred expressions in assignment
```

Python в любой момент времени изменяет тип переменной в соответствии с данными, хранящимися в ней. Пример:

```
a = 'Строка'  # Тип str
a = 7 # Теперь переменная имеет тип int
```

Определить, на какой тип данных ссылается переменная, позволяет функция **type (<*имя переменной*>)**:

```
type(a)
<class 'int'>
```

Проверить тип данных, хранящихся в переменной, можно следующими способами:

– сравнить значение, возвращаемое функцией **type ()**, с названием типа данных:

```
x = 10
if type(x) == int:
    print('Это тип int')
Это тип int
```

– проверить тип с помощью функции **isinstance()**:

```
s = 'Строка'
if isinstance(s, str):
    print('Это тип str')
Это тип str
```

4.11 Математические операторы в языке Python

Производить операции над числами позволяют следующие операторы:

– + (сложение):

```
10 + 5 # Целые числа
15
12.4 + 5.3 # Вещественные числа
17.7
10 + 22.4 # Целое и вещественное числа
32.4
```

– - (вычитание):

```
10 - 5 # Целые числа
5
12.4 - 5.2 # Вещественные числа
7.2
12 - 5.2 # Целое и вещественное числа
6.8
```

– * (умножение):

```
10 * 5 # Целые числа
50
12.4 * 5.3 # Вещественные числа
65.72
10 * 22.4 # Целое и вещественное числа
224.0
```

– / (деление). Результатом деления всегда является вещественное число, даже если производится деление целых чисел. Примеры:

```
10 / 5 # Целые числа
2.0
10 / 3 # Целые числа
3.3333333333333335
10.0 / 5.0 # Вещественные числа
2.0
10.0 / 3.0 # Вещественные числа
3.3333333333333335
10 / 5.0 # Деление целого на вещественное
2.0
10.0 / 5 # Деление вещественного на целое
2.0
```

– // (деление с округлением вниз). Вне зависимости от типа чисел остаток отбрасывается. Примеры:

```
10 // 5 # Деление целых чисел без остатка
2
10 // 3 # Деление целых чисел с остатком
3
10.0 // 5.0 # Вещественные числа
2.0
10.0 // 3.0 # Вещественные числа
3.0
10 // 5.0 # Деление целого на вещественное
```

```
2.0
10 // 3.0 # Деление целого на вещественное
3.0
10.0 // 5 # Деление вещественного на целое
2.0
10.0 // 3 # Деление вещественного на целое
3.0
```

– % (остаток от деления):

```
10 % 5 # Деление целых чисел без остатка
0
10 % 3 # Деление целых чисел с остатком
1
10.0 % 5.0 # Вещественные числа
0.0
10.0 % 3.0 # Вещественные числа
1.0
10 % 5.0 # Целые и вещественные числа
0.0
10 % 3.0 # Целые и вещественные числа
1.0
10.0 % 5 # Вещественные и целые числа
0.0
10.0 % 3 # Вещественные и целые числа
1.0
```

– ** (возведение в степень):

```
10 ** 2, 10.0 ** 2
(100, 100.0)
```

– унарный минус (-) и унарный плюс (+):

```
+10, +10.0, -10, -10.0 -(-10), -(-10.0)
(10, 10.0, -10, 0.0, 10.0)
```

Как видно из примеров, операции над числами разных типов возвращают число, имеющее более сложный тип из типов, участвующих в операции. Целые числа имеют самый простой тип, далее идут вещественные числа и самый сложный тип - комплексные числа. Таким образом, если в операции участвуют целое число и вещественное, то целое число будет автоматически преобразовано в вещественное число, а затем произведена операция над вещественными числами. Результатом этой операции станет вещественное число.

При выполнении операций над вещественными числами следует учитывать ограничения точности вычислений. Например, результат следующей операции может показаться странным:

```
0.3 - 0.1 - 0.1 - 0.1
-2.7755575615628914e-17
```

Ожидаемым был бы результат 0.0, но, как видно из примера, получен совсем другой результат. Если необходимо производить операции с фиксированной точностью, то следует использовать модуль **decimal**:

```
from decimal import Decimal
Decimal('0.3') - Decimal('0.1') - Decimal('0.1') - Decimal('0.1')
Decimal('0.0')
```

4.12 Двоичные операторы в языке Python

Побитовые операторы предназначены для манипуляции отдельными битами. Язык **Python** поддерживает следующие побитовые операторы:

- ~ (двоичная инверсия). Значение каждого бита заменяется на противоположное:

```
x = 100 # 01100100
x = ~x  # 10011011
print(x)
-101
```

- & (двоичное И):

```
x = 100   # 01100100
y = 75    # 01001011
z = x & y # 01000000
'{0:b} & {1:b} = {2:b}'.format(x, y, z)
'1100100 & 1001011 = 1000000'
print(z)
64
```

- | (двоичное ИЛИ):

```
x = 100   # 01100100
y = 75    # 01001011
z = x | y # 01101111
'{0:b} | {1:b} = {2:b}'.format(x, y, z)
'1100100 | 1001011 = 1101111'
print(z)
111
```

- ^ (двоичное исключающее ИЛИ):

```
x = 100   # 01100100
y = 250   # 11111010
z = x ^ y # 10011110
'{0:b} ^ {1:b} = {2:b}'.format(x, y, z)
'1100100 ^ 11111010 = 10011110'
print(z)
158
```

- << (сдвиг влево). Сдвигает двоичное представление числа влево на один или более разрядов и заполняет разряды справа нулями:

```
x = 100    # 01100100
y = x << 1 # 11001000
print(y)
200
z = y << 1 # 10010000
```

```
print(z)
400
k = z << 2 # 01000000
print(k)
1600
```

– >> (сдвиг вправо). Сдвигает двоичное представление числа вправо на один или более разрядов и заполняет разряды слева нулями, если число положительное:

```
x = 100    # 01100100
y = x >> 1 # 00110010
print(y)
50
x = y >> 1 # 00011001
print(x)
25
k = x >> 2 # 00000110
print(k)
6
```

Если число отрицательное, то разряды слева заполняются единицами:

```
x = -127   # 10000001
y = x >> 1 # 11000000
print(y)
-64
z = y >> 2 # 11110000
print(z)
-16
k = z << 1 # 11100000
print(k)
-32
m = k >> 1 # 11110000
print(m)
-16
```

4.13 Операторы для работы с последовательностями в языке Python

Для работы с последовательностями предназначены следующие операторы:

– + (конкатенация):

```
print('Строка1' + 'Строка2') # Строки
Строка1Строка2
[1, 2, 3] + [4, 5, 6]       # Списки
[1, 2, 3, 4, 5, 6]
(1, 2, 3) + (4, 5, 6)       # Кортежи
(1, 2, 3, 4, 5, 6)
```

– * (повторение):

```
's' * 20        # Строки
'ssssssssssssssssssss'
[1, 2] * 3      # Списки
[1, 2, 1, 2, 1, 2]
 (1, 2) * 3     # Кортежи
(1, 2, 1, 2, 1, 2)
```

– in (проверка на вхождение). Если элемент входит в последовательность, то возвращается логическое значение **True**:

```
"Строка" in "Строка для поиска"  # Строки
True
"Строка2" in "Строка для поиска" # Строки
False
2 in [1, 2, 3], 4 in [1, 2, 3]   # Списки
(True, False)
2 in (1, 2, 3), 6 in (1, 2, 3)   # Кортежи
(True, False)
```

– not in (проверка на невхождение). Если элемент не входит в последовательность, возвращается **True**:

```
"Строка" not in "Строка для поиска"    # Строки
False
"Строка2" not in "Строка для поиска"   # Строки
True
2 not in [1, 2, 3], 4 not in [1, 2, 3] # Списки
(False, True)
2 not in (1, 2, 3), 6 not in (1, 2, 3) # Кортежи
(False, True)
```

4.14 Операторы присваивания в языке Python

Операторы присваивания предназначены для сохранения значения в переменной. Перечислим операторы присваивания, доступные в языке **Python**:

– = - присваивает переменной значение:

```
x = 5
print(x)
5
```

– += - увеличивает значение переменной на указанную величину:

```
x = 5
x += 10 # Эквивалентно x = x + 10
print(x)
15
```

Для последовательностей оператор += производит конкатенацию:

```
s = "Стр"
s += "ока"
print(s)
'Строка'
```

– -= - уменьшает значение переменной на указанную величину:

```
x = 10
x -= 5 # Эквивалентно x = x - 5
print(x)
```

```
5
```

– *= - умножает значение переменной на указанную величину:

```
x = 10
x *= 5 # Эквивалентно x = x * 5
print(x)
```

```
50
```

Для последовательностей оператор ***=** производит повторение:

```
s = '*'
s *= 20
print(s)
```

```
'********************'
```

– /= - делит значение переменной на указанную величину:

```
x = 10
x /= 3 # Эквивалентно x = x / 3
print(x)
```

```
3.3333333333333335
```

```
x = 10
x /= 3.0 # Эквивалентно x = x / 3
print(x)
```

```
3.3333333333333335
```

– //= - деление с округлением вниз и присваиванием:

```
x = 10
x //= 3 # Эквивалентно x = x // 3
print(x)
```

```
3
```

```
y = 10.0
y //= 3.0 # Эквивалентно y = y // 3.0
print(y)
```

```
3.0
```

– %= - деление по модулю и присваивание:

```
x = 10
x %= 2 # Эквивалентно x = x % 2
print(x)
```

```
0
```

```
y = 10
y %= 3 # Эквивалентно y = y % 3
print(y)
```

```
1
```

– **= - возведение в степень и присваивание:

```
x = 10
x **= 2 # Эквивалентно x = x ** 2
```

```
print(x)
100
```

4.15 Приоритет выполнения операторов в языке Python

В какой последовательности будет вычисляться приведенное далее выражение?

```
x = 5 + 10 * 3 / 2
```

Это зависит от приоритета выполнения операторов. В данном случае последовательность вычисления выражения будет такой:

1 Число 10 будет умножено на 3, т. к. приоритет оператора умножения выше приоритета оператора сложения.

2 Полученное значение будет поделено на 2, т. к. приоритет оператора деления равен приоритету оператора умножения (а операторы с равными приоритетами выполняются слева направо), но выше, чем у оператора сложения.

3 К полученному значению будет прибавлено число 5, т. к. оператор присваивания = имеет наименьший приоритет.

4 Значение будет присвоено переменной x.

```
x = 5 + 10 * 3 / 2
print(x)
```

С помощью скобок можно изменить последовательность вычисления выражения:

```
x = (5  + 10) * 3 / 2
```

Теперь порядок вычислений станет иным:

1 К числу 5 будет прибавлено 10.

2 Полученное значение будет умножено на 3.

3 Полученное значение будет поделено на 2.

4 Значение будет присвоено переменной x.

```
x = (5 + 10) * 3 / 2
print(x)
22.5
```

Перечислим операторы в порядке убывания приоритета:

1 -x, +x, ~x, ** - унарный минус, унарный плюс, двоичная инверсия, возведение в степень. Если унарные операторы расположены слева от оператора **, то возведение в степень имеет больший приоритет, а если справа - то меньший. Например, выражение:

```
2  -10  ** -2
```

эквивалентно следующей расстановке скобок:

```
-(10 ** (-2))       .
```

3 *, %, /, // - умножение (повторение), остаток от деления, деление, деление с округлением вниз.

4 +, - - сложение (конкатенация), вычитание.

5 <<, >> - двоичные сдвиги.

6 & - двоичное И.

7 ^ - двоичное исключающее ИЛИ.

8 | - двоичное ИЛИ.
9 =, +=, -=, *=, /=, //=, %=, **= - присваивание.

4.16 Математические функции модули Math в языке Python

В связи с невозможностью записи некоторых стандартных математических функций с клавиатуры персонального компьютера в языке Python существуют так называемые «строенные функции, с помощью которых пользователь записывает арифметические выражения. Некоторые математические функции языка программирования Python представлены в таблице 1.

Прежде чем использовать математические функции, необходимо в начале программы написать инструкцию import math, однако тогда перед упоминанием каждой функция необходимо будет добавлять имя модуля math, например, y=math.sin(x). Другой способ, который позволил избежать многократного вызова модуля math, сделать следующую запись в начале программы: from math import *.

Таблица 1 – Общие математические функции модуля Math

Функция	**Запись на Python**	**Действие**
1	2	3
sin x	math.sin (x)	Возвращает значение функции Sin от числа x
cos x	math.cos (x)	Возвращает значение функции Cos от числа x
tg x	math.tan (x) или math.sin (x) / math.cos (x)	Возвращает значение функции Tg от числа x
ctg x	math.cos (x)/ math.sin (x)	Возвращает значение функции Ctg от числа x
|x|	math.abs (x)	Возвращает абсолютную величину числа x
e^x	math.exp (x)	Возвращает результат возведения числа **e** в степень x
ln x	math.log10(x)	Возвращает натуральный логарифм от x+1
$\sqrt{x}$	math.sqrt (x)	Возвращает результат извлечения квадратного корня числа x
$\log_{10} x$	math.log(x)	Возвращает логарифм числа x по основанию 10
$\cos^2 x$	math.cos (x)*math.cos (x)	Возвращает результат возведения функции Cos x в квадрат
acos x	math.acos (x)	Возвращает значение функции арккосинус от числа x
asin x	math.asin (x)	Возвращает значение функции арксинус от числа x
atan x	math.atan (x)	Возвращает значение функции арктангенс от числа x
pi	pi	Возвращает 3.141592653589793
degrees (x)	math.degrees(x)	Преобразует радианы в градусы
radians(x)	math.radians(x)	Преобразует градусы в радианы
floor (x)	math.floor (x)	Возвращает значение, округленное до ближайшего меньшего целого
ceil(x)	math.ceil(x)	Возвращает значение, округленное до ближайшего большего целого
factorial(x)	math.factorial(x)	Возвращает факториал числа 3!=1*2*3

В таблице 2 представлены некоторые встроенные функции для работы с числами, не требующие подключения модуля math.

Таблица 2 – Функция для работы с числами

Функция	Запись на Python	Описание
round (x)	round (x)	Возвращает результат округления числа x до ближайшего меньшего целого значения для чисел с дробной частью меньше 0,5 или результат округления числа x до ближайшего большего целого значения для чисел с дробной частью больше 0,5
pow (x, y)	pow(x, y) другой вариант x**y	Возвращает результат возведения числа x в степень y
max (x, y)	max (список чисел через запятую)	Возвращает большее значение из списка чисел
min (x, y)	min (список чисел через запятую)	Возвращает меньшее значение из списка чисел
sum (x, y)	sum (список чисел через запятую)	Возвращает сумму значений элементов последовательности
int (объект)	int (объект)	Преобразует объект (например, строковое значение, дробное значение) в целое число
float (объект)	float (объект)	Преобразует объект (например, строковое значение, целое значение) и вещественное число

Python предоставляет возможность совершить операции нал числами, если программист укажет исходные тайные (операнды) и математические действия в операторе print, например, инструкция print(2016+40-20) выведет на экран число 2036.

4.17 Пошаговая установка PyCharm

JetBrains PyCharm — это интегрированная среда разработки (IDE), используемая для написания программ на Python. В ней есть инструменты для анализа кода, графический отладчик, встроенное модульное тестирование и поддержка веб-разработки с Django.

Для установки PyCharm сперва нужно установить интерпретатор Python. Дальше — пошаговый процесс установки PyCharm.

1 Перейдите на сайт JetBrains https://www.jetbrains.com/pycharm/. PyCharm — это проект JetBrains, который представлен на рисунке 4.



Рисунок 4 – Сайт

2 Нажмите «Download». Произойдет перенаправление на страницу загрузки PyCharm (рисунок 5).

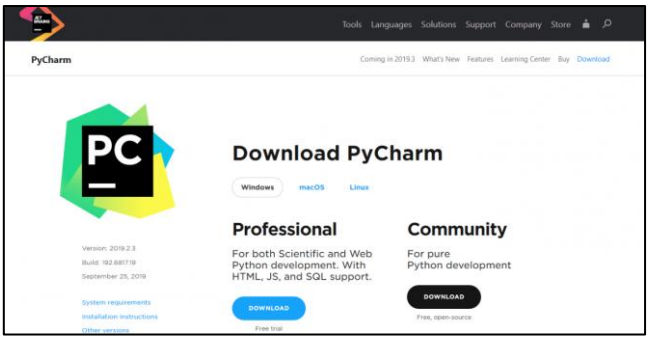


Рисунок 5 – Скачать программу

3 Дальше необходимо выбрать версию PyCharm: **Community** или **Professional**. Professional — это платная версия с полным набором функций. Она идеально подходит для профессиональной разработки. Community бесплатная. Ею можно пользоваться благодаря набору базовых возможностей. После нажатия на кнопку «Загрузить» скачивание начнется автоматически (рисунок 6).



Рисунок 6 – Благодарим за загрузку

4 Теперь необходимо запустить установку (~ pycharm-community-2023.3.4). Нажмите «Next» (рисунок 7).



Рисунок 7 – Настройка программы

При установке используется версия PyCharm 2023.3.4, но может быть выпущена новая версия. Учитывайте это, часть 2023.3.4может быть любой.

5 Оставьте папку установки по умолчанию. Это папка C:\Program files(x86)\JetBrains\PyCharm Community Edition 2023.3.4. Если выбрана другая, лучше указать именно это направление. Нажмите «Next».

6 Поставьте галочки напротив обоих пунктов, если нужен ярлык для рабочего стола. Нажмите «Next» (рисунок 8).

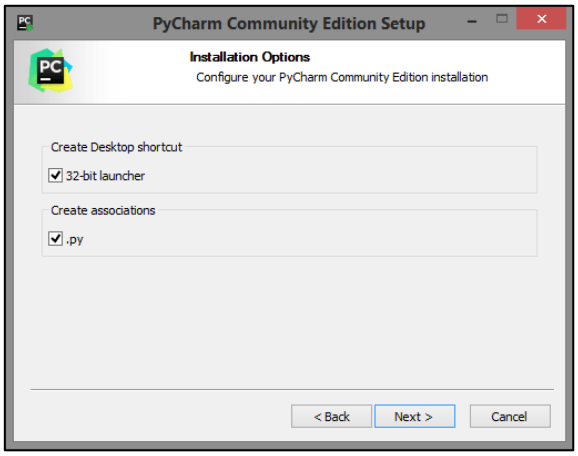


Рисунок 8 – Варианты установки

7 Далее установщик попросит указать имя для отображения в стартовом меню. По умолчанию это JetBrains. Нажмите «Install».

8 Установка начнется автоматически. Дождитесь завершения.

9 После завершения установки поставьте галочку напротив «Run PyCharm Community Edition» для запуска, а затем «Finish» (рисунок 9).

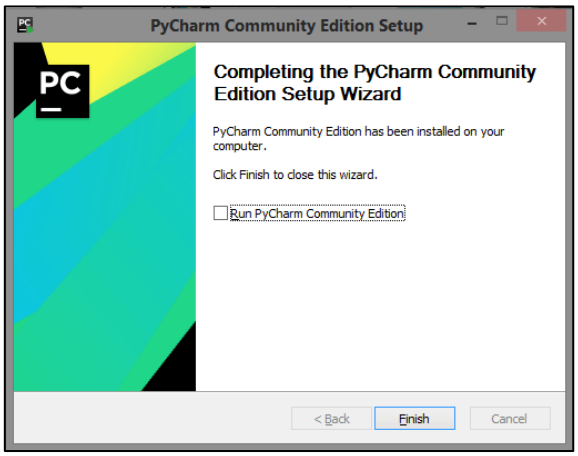


Рисунок 9 – Запустите программу

10 Если вы только учите язык Python и установили PyCharm впервые, тогда выберите пункт «I do not have a previous version of PyCharm or I do not want to import my settings» и «ОК».

11 PyCharm запустится. Первый раз этой займет больше времени.

12 После приветственного окна нажмите «Create New Project».

13 Выберите директорию для сохранения проекта. Во втором поле нужно указать местоположение интерпретатора Python (который уже должен быть установлен). Обычно PyCharm находит его самостоятельно. Если не удалось, нужно указать путь к нему и нажать «Create» (рисунок 10).

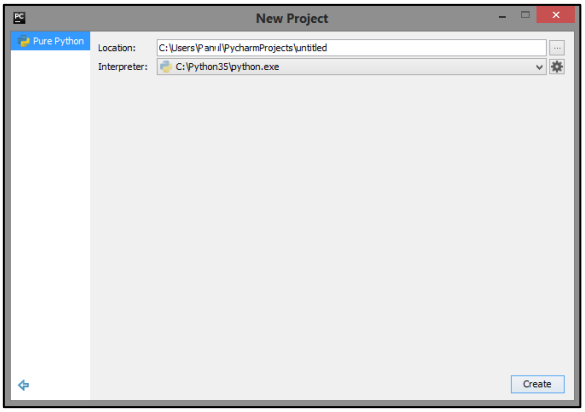


Рисунок 10 – Новый проект

14 Нажмите «File > New > Python File». Введите название нового файла. Он откроется с чистой консолью. Можно начинать писать программу для проверки. Напишите что-то простое: например, программу, которая выводит «Hello World!» на экран. Используйте этот код, чтобы проверить работу PyCharm.

```
print("Hello World!")
```

В любом месте кликните правой кнопкой мыши и выберите «Run». Программа выполнится, а вывод будет показан на экране. Это значит, что PyCharm установлен и готов к работе. Удачи и успехов в изучении языка программирования Python.

4.18 Пример решения задачи на языке программирования Python

Задача. Вычислите значения арифметических выражений и выведите на экран результаты вычислений.

Исходные данные: x =1,4444; b=0,318; t=2,1; f=1,3.

$$y = 9x^2 + \sin^2 x\sqrt{a-b}$$

$$z=\sqrt[3]{x^t}\left(ax^3-\frac{x^2}{2!}\right)$$

Запишем арифметические выражения на языке программирования.

y=9*x*x+sin(x)*sin(x)*sqrt(a-b)

z=exp(1/3*log(pow(x,t)))*((a*x*x*x-(x*x)/(1*2)))

Разработка алгоритма решения задачи представлена на рисунке 11.

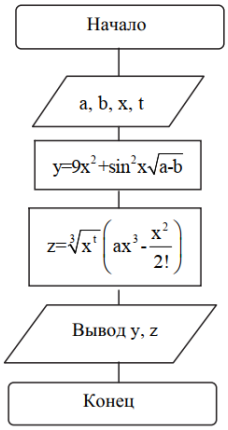


Рисунок 11 – Алгоритм решения задачи

Мы ввели дополнительную переменную z1 для вычисления значения x, возведенного в степень t, для более короткой записи в выражении, вычисляющем значение z. Оператор exit(0) вызовет появление окна сообщения, в котором спрашивается о том, хотите ли вы завершить запущенный на выполнение процесс (программу). В случае положительного ответа программа прекратит свою работу и, кроме того, закроется интерактивная сессия Python.

Ограничить количество знаков после точки в вещественной записи числа поможет метод format. С самим понятием «метод» мы познакомимся позже, а сейчас посмотрим, как применить его к выводу данных для того, чтобы результат стал более читаемым.

Метод format записывается сразу после точки и применяется к результирующей ячейке, для которой мы хотим уменьшить количество знаков. В фигурных скобках указывается так называемый спецификатор формата. 0 – это поле, определяющее индекс позиции, при этом нумерация начинается с нуля (можно задать ширину поля, например 0:10, тогда выводимое число будет «отодвинуто» вправо на 10 позиций), .3 – точность вычислений, т. е. количество знаков после точки, f – вещественное число в десятичном представлении.

Решение задачи:

```
from math import *
a=float (input ("Введите значение a"))
b=float(input("Введите значение b"))
x=float(input("Введите значение x"))
t=float(input("Введите значение t"))
y=9*x*x+sin(x)*sin(x)*sqrt(a-b)
z1= log(pow(x,t))
z= pow(z1,(1/3))*((a*x*x*x-(x*x)/(1*2)))
print("\nЗначение y = ",'{0:.3f}'.format(y))
print("\nЗначение z = ",'{0:.3f}'.format(z))
exit(0)
```

5 Индивидуальное задание

5.1 Установите программу PyCharm Community и новую версию языка программирования Python.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом для расчета по двум формулам. Предварительно подготовьте тестовые примеры по второй формуле с помощью калькулятора (результат вычисления по первой формуле должен совпадать со второй). Выполните отладку и результаты вычислений выведите на экран:

- в формате с плавающей точкой, ограничить количество знаков после запятой до трех.

1	$a_1 = \cos a + \sin a + \cos 3a + \sin 3a;$	$a_2 = \frac{1}{4} - \frac{1}{4}\sin\left(\frac{5}{2}\pi - 8a\right).$
2	$a_1 = \cos a + \sin a + \cos 3a + \sin 3a;$	$a_2 = 2\sqrt{2}\cos a \cdot \sin\left(\frac{\pi}{4} + 2a\right).$
3	$a_1 = \frac{\sin 2a + \sin 5a - \sin 3a}{\cos a + 1 - 2\sin^2 2a};$	$a_2 = 2\sin a$
4	$a_1 = \frac{\sin 2a + \sin 5a - \sin 3a}{\cos a - \cos 3a + \cos 5a};$	$a_2 = \mathrm{tg}\,3a.$
5	$a_1 = 1 - \frac{1}{4}\sin^2 2a + \cos 2a;$	$a_2 = \cos^2 a + \cos^4 a.$
6	$a_1 = \cos a + \cos 2a + \cos 6a + \cos 7a;$	$a_2 = 4\cos\frac{a}{2} \cdot \cos\frac{5}{2}a \cdot \cos 4a.$
7	$a_1 = \cos^2\left(\frac{3}{8}\pi - \frac{a}{4}\right) - \cos^2\left(\frac{11}{8}\pi + \frac{a}{4}\right);$	$a_2 = \frac{\sqrt{2}}{2}\sin\frac{a}{2}.$
8	$a_1 = \cos^4 x + \sin^2 y + \frac{1}{4}\sin^2 2x - 1;$	$a_2 = \sin(y + x) \cdot \sin(y - x).$
9	$a_1 = (\cos a - \cos\beta)^2 - (\sin a - \sin\beta)^2;$	$a_2 = -4\sin^2\frac{a-\beta}{2} \cdot \cos(a + \beta).$
10	$a_1 = \frac{\sin\left(\frac{\pi}{2} + 3a\right)}{1 - \sin(3a - \pi)};$	$a_2 = \mathrm{ctg}\left(\frac{5}{4}\pi + \frac{3}{2}a\right).$
11	$a_1 = \cos 18a - 2\sin 3a + \mathrm{ctg}\,5a;$	$a_2 = \sin\left(\frac{5}{2}\pi - 9a\right).$
12	$a_1 = \cos a - 2 \cdot \cos 3a$	$a_2 = 2\sqrt{2}\sin\left(\frac{\pi}{3} + 2a\right).$
13	$a_1 = \frac{\sin 4a - \sin 3a}{\cos a + \sin^2 2a};$	$a_2 = 4\sin 6a.$
14	$a_1 = \sin 4a + \sin 2a;$	$a_2 = 2\sin 3a \cdot \cos a.$
15	$a_1 = \cos^2 a - \sin^2 a;$	$a_2 = \cos 2a.$

5.3 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Вычислите площадь и объем сферы.

$$S_{сф} = 4\pi R^2; V_{сф} = 4/3\pi R^3$$

2 Подсчитайте количество часов, минут и секунд в заданном числе суток.

ch=24*S
min=60*ch
sec=60*min

3 Вычислите площадь треугольника по трем сторонам. Вычисление проводиться по формуле Герона.

$S=\sqrt{p(p-a)(p-b)(p-c)}$, где S – площадь треугольника; p=(a+b+c)/2 – полупериметр; a, b, c – длина стороны треугольника.

4 Найдите решение системы линейных уравнений:

$$\begin{cases} a_1x + b_1y = c_1 \\ a_2x + b_2y = c_2 \end{cases}$$

По правилу Крамера:

$$x = \frac{c_1b_2 - c_2b_1}{a_1b_2 - a_2b_1}.$$

5 Определите объемы цилиндра и конуса с радиусом основания R=5см и высотой h=8 см.

$$V_ц = h\pi R^2;\ V_k = \frac{1}{3}h\pi R^2$$

6 Замените местами значения двух переменных, введенных с клавиатуры.

7 Вычислите площадь и периметр:

- треугольника по данным трем сторонам;
- прямоугольника по данным ширине и высоте;
- круга по заданному радиусу.

8 Найдите сумму и произведение цифр трехзначного числа, которое вводит пользователь.

9 Запишите уравнение прямой, проходящей через две точки. Координаты, точек вводятся с клавиатуры.

10 Вычислите длину гипотенузы прямоугольного треугольника по двум его катетам. Значения катетов вводятся с клавиатуры.

11 Составьте программу вычисления длины окружности L=2*π*R и площади круга S=π*R заданного радиуса R.

12 Даны два числа a и b. Найти их среднее арифметическое и среднее геометрическое.

13 Вычислите площадь и периметр прямоугольника по данным ширине и высоте.

14 Вычислите расстояние между двумя точками на плоскости по заданным координатам.

15 Найдите сумму и разность квадратов двух чисел, введённых пользователем.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Дайте определение понятию «Линейный алгоритм» в языке программирования Python.

7.2 Назовите и поясните способы описания алгоритмов в языке программирования Python.

7.3 Охарактеризуйте функциональные блоки, используемые в блок-схемах. Поясните их назначение в языке программирования Python.

7.4 Опишите типы данных в языке программирования Python.

7.5 Объясните, как в языке программирования Python осуществляется ввод и вывод данных.

7.6 Охарактеризуйте инструкции, которые необходимо прописывать в программах, написанных на языке программирования Python, для использования в них математических функций?

7.7 Опишите особенность динамической типизации, используемой в языке программирования Python?

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.